

Persistencia en .NET

SQL Server + ADO.NET

Material de trabajo:

- SSMS
- Aplicación de consola .NET 10
- Microsoft.Data.SqlClient
- Caso práctico BiblioNET.



SSMS

.NET 10

ADO.NET

BiblioNET

Objetivo y material de la sesión

02

Qué se busca reforzar y con qué herramientas trabajaremos.

Objetivo

Comprender cómo se modelan y persisten datos en SQL Server, y cómo una aplicación .NET interactúa con ellos mediante ADO.NET.

Durante la sesión veremos

- Modelo relacional
- SQL básico: DDL y DML
- Índices y restricciones
- SqlConnection, SqlCommand y SqlDataReader
- SQL directo y Stored Procedures

Material

SQL Server Management Studio

crear y consultar la base

BiblioNET.Consola

ejecutar operaciones desde C#

Scripts SQL

tablas, índices y procedimientos

Visual Studio

depurar el flujo de ejecución

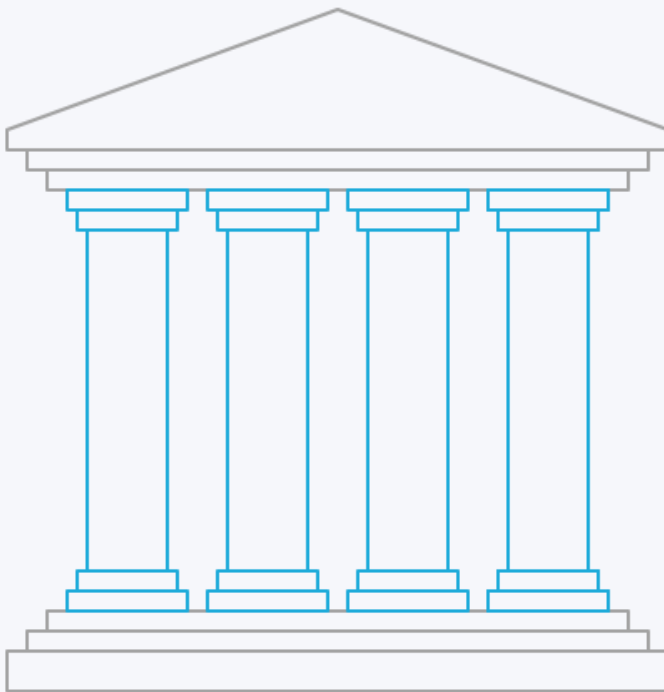


¿Qué entendemos por persistencia?

03

Persistir no es solo guardar datos; también implica consistencia y reglas.

Conceptos de Persistencia



Supervivencia estructurada

Garantizar la supervivencia de datos de manera estructurada y confiable.



Gestión de información

Capacidad de guardar, consultar y modificar información de forma duradera.



Integridad y consistencia

Mantener reglas de integridad y consistencia en los datos.



Implementación técnica

App .NET utilizando acceso a datos con SQL Server.

En esta sesión nos enfocamos en las dos piezas más cercanas al dato: diseño en SQL Server y acceso directo desde C#.

- Diseñar tablas antes de escribir código.
- Declarar reglas en la base de datos.
- Ejecutar operaciones desde una app .NET.
- Validar el resultado directamente en SQL Server.

SQL Server: conceptos fundamentales

04

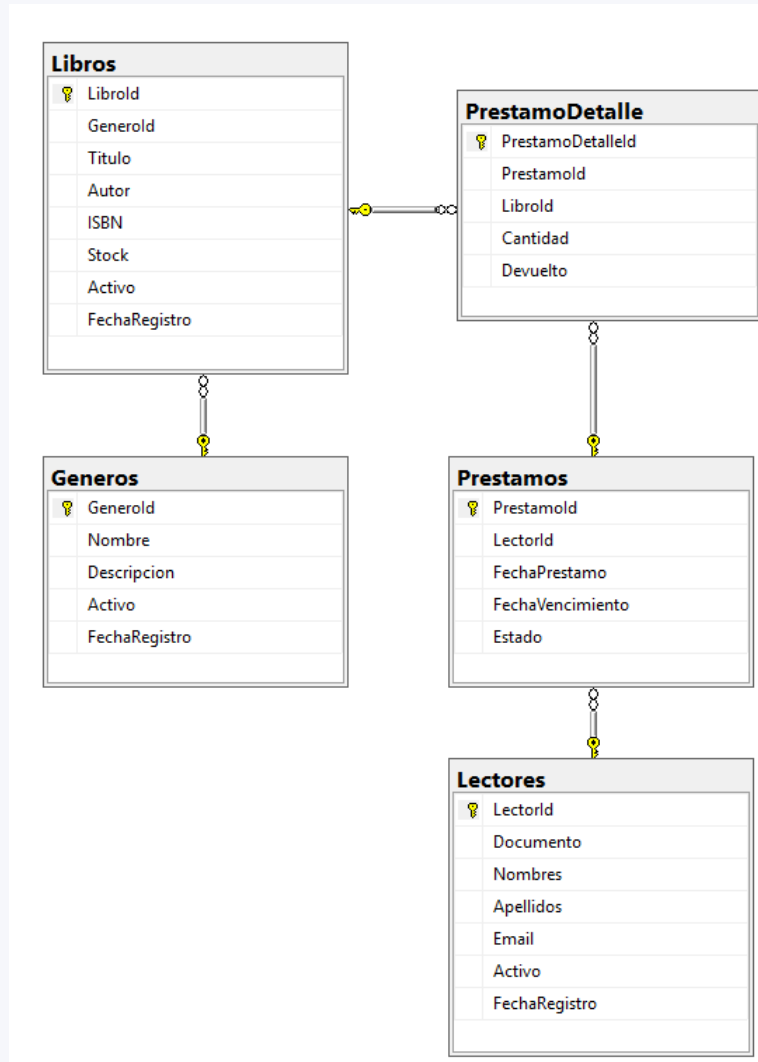
Vocabulario mínimo para modelar datos antes de consumirlos desde .NET.

Concepto	Descripción
 Base de datos	Contenedor lógico donde se organiza la información.
 Tabla	Estructura formada por filas y columnas.
 Primary Key (PK)	Identifica de forma única cada registro.
 Foreign Key (FK)	Relaciona tablas y protege la integridad referencial.
 Restricciones	Reglas como NOT NULL, UNIQUE, CHECK y DEFAULT.
 Índice	Estructura auxiliar utilizada para optimizar consultas.

Modelo relacional de BiblioNET

05

El caso práctico se basa en cinco entidades relacionadas.



Entidades del caso

- Generos
- Libros
- Lectores
- Prestamos
- PrestamoDetalle

Este modelo permite explicar catálogo, actores, operaciones y detalle de una operación.

DER real generado desde SQL Server

Relaciones e integridad referencial

06

Las FK conectan tablas y evitan referencias inexistentes.

Generos

1 : N

Libros

Un género puede clasificar muchos libros.

Lectores

1 : N

Prestamos

Un lector puede registrar muchos préstamos.

Prestamos

1 : N

PrestamoDetalle

Un préstamo puede contener varios libros.

Libros

1 : N

PrestamoDetalle

Un libro puede aparecer en muchos detalles.

¿Por qué importa?

La integridad referencial impide que la aplicación guarde datos incoherentes, como un libro con un género inexistente.

Índices: rendimiento con un costo

07

Un índice ayuda cuando responde a una consulta frecuente; no se crea “por si acaso”.

Índices de base de datos



Cuándo tiene sentido

- Filtras por una columna con frecuencia.
- Haces JOIN usando una FK.
- Ordenas o buscas por rangos.
- La tabla crecerá y la consulta se repetirá.

```
CREATE INDEX IX_Libros_GeneroId ON dbo.Libros(GeneroId);  
CREATE INDEX IX_Libros_Titulo ON dbo.Libros(Titulo);
```

Costo esperado: más espacio y más trabajo al insertar, actualizar o eliminar.

SQL: definir y manipular datos

08

Antes de ADO.NET, identificamos qué tipo de operación estamos ejecutando.

Comandos SQL



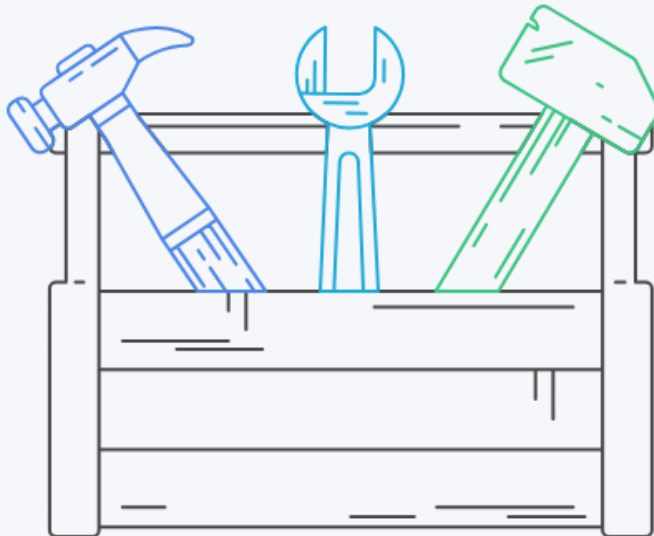
DML

Permite modificar la información almacenada.



DDL

Permite definir o modificar estructuras.



Consulta de datos

Comando para realizar consultas de datos.

```
CREATE TABLE dbo.Libros
(
    LibroId          INT IDENTITY(1,1) NOT NULL,
    GeneroId         INT NOT NULL,
    Titulo           NVARCHAR(150) NOT NULL,
    Autor            NVARCHAR(120) NOT NULL,
    ISBN             VARCHAR(20) NOT NULL,
    Stock            INT NOT NULL CONSTRAINT DF_Libros_Stock DEFAULT (0),
    Activo           BIT NOT NULL CONSTRAINT DF_Libros_Activo DEFAULT (1),
    FechaRegistro    DATETIME2(0) NOT NULL
    CONSTRAINT DF_Libros_FechaRegistro DEFAULT (SYSDATETIME()),

    CONSTRAINT PK_Libros PRIMARY KEY (LibroId),
    CONSTRAINT UQ_Libros_ISBN UNIQUE (ISBN),
    CONSTRAINT FK_Libros_Generos
        FOREIGN KEY (GeneroId) REFERENCES dbo.Generos(GeneroId),
    CONSTRAINT CK_Libros_Stock CHECK (Stock >= 0)
);
```

ADO.NET: acceso a datos desde .NET

09

ADO.NET nos permite comunicarnos directamente con SQL Server desde C#.

Componentes que usaremos

SqlConnection abre el canal hacia SQL Server

SqlCommand define qué se ejecutará

SqlParameter envía valores como datos

SqlDataReader lee filas secuencialmente

Idea clave

ADO.NET da mucho control, pero exige escribir más código y hacer mapping manual.



SqlConnection: abrir, utilizar y liberar

10

Centralizamos la configuración, no mantenemos una conexión global abierta.

Patrón recomendado

- Crear la conexión desde una fábrica.
- Abrirla justo antes de usarla.
- Liberarla con using / await using.
- Dejar que SqlConnection gestione el pool de conexiones.

No buscamos una SqlConnection estática abierta durante toda la aplicación.

```
private const string ConnectionString =  
    "Server=localhost;" +  
    "Database=BiblioNETDB;" +  
    "Integrated Security=True;" +  
    "TrustServerCertificate=True;";  
  
public static SqlConnection Create()  
{  
    return new SqlConnection(ConnectionString);  
}
```

Connection Pooling

Al liberar una SqlConnection, normalmente la conexión física vuelve al pool y puede reutilizarse.

SqlCommand, parámetros y lectura de resultados

11

El comando ejecuta la operación; el reader permite recorrer los resultados y realizar el mapping hacia objetos C#.

Mensaje de seguridad

No concatenar valores de usuario dentro del SQL. Usar parámetros para enviar datos de forma explícita.

```
const string sql = ""
    SELECT
        l.LibroId,
        l.GeneroId,
        g.Nombre AS Genero,
        l.Titulo,
        l.Autor,
        l.ISBN,
        l.Stock,
        l.Activo
    FROM dbo.Libros AS l
    INNER JOIN dbo.Generos AS g
        ON g.GeneroId = l.GeneroId
    WHERE l.Activo = 1
    ORDER BY l.Titulo;
"";

var libros = new List<Libro>();

await using var connection = DbConnectionFactory.Create();
await connection.OpenAsync();

await using var command = new SqlCommand(sql, connection)
{
    CommandType = CommandType.Text
};
```

SQL directo vs Stored Procedures

12

Dos formas válidas de ejecutar operaciones desde ADO.NET.

¿Cómo debe interactuar su aplicación .NET con SQL Server?



SQL Directo

1

Envía un SqlCommand que contiene una sentencia SQL parametrizada directamente a la base de datos.

Ejecuta lógica de consulta en bruto

2

Se basa en la capa de aplicación para definir y administrar la sintaxis SQL específica.

La lógica reside en el código de la aplicación



Procedimientos Almacenados

1

Envía un SqlCommand que contiene solo el nombre del procedimiento y los parámetros para activar la lógica predefinida del lado del servidor.

Invoca rutinas de base de datos precompiladas

2

Descarga la lógica de ejecución al servidor de base de datos, que luego interactúa directamente con las tablas.

La lógica reside en la capa de base de datos

SQL directo

La sentencia SQL está definida en la aplicación.

```
new SqlCommand(sql, connection)
{
    CommandType = CommandType.Text
};
```

Stored Procedure

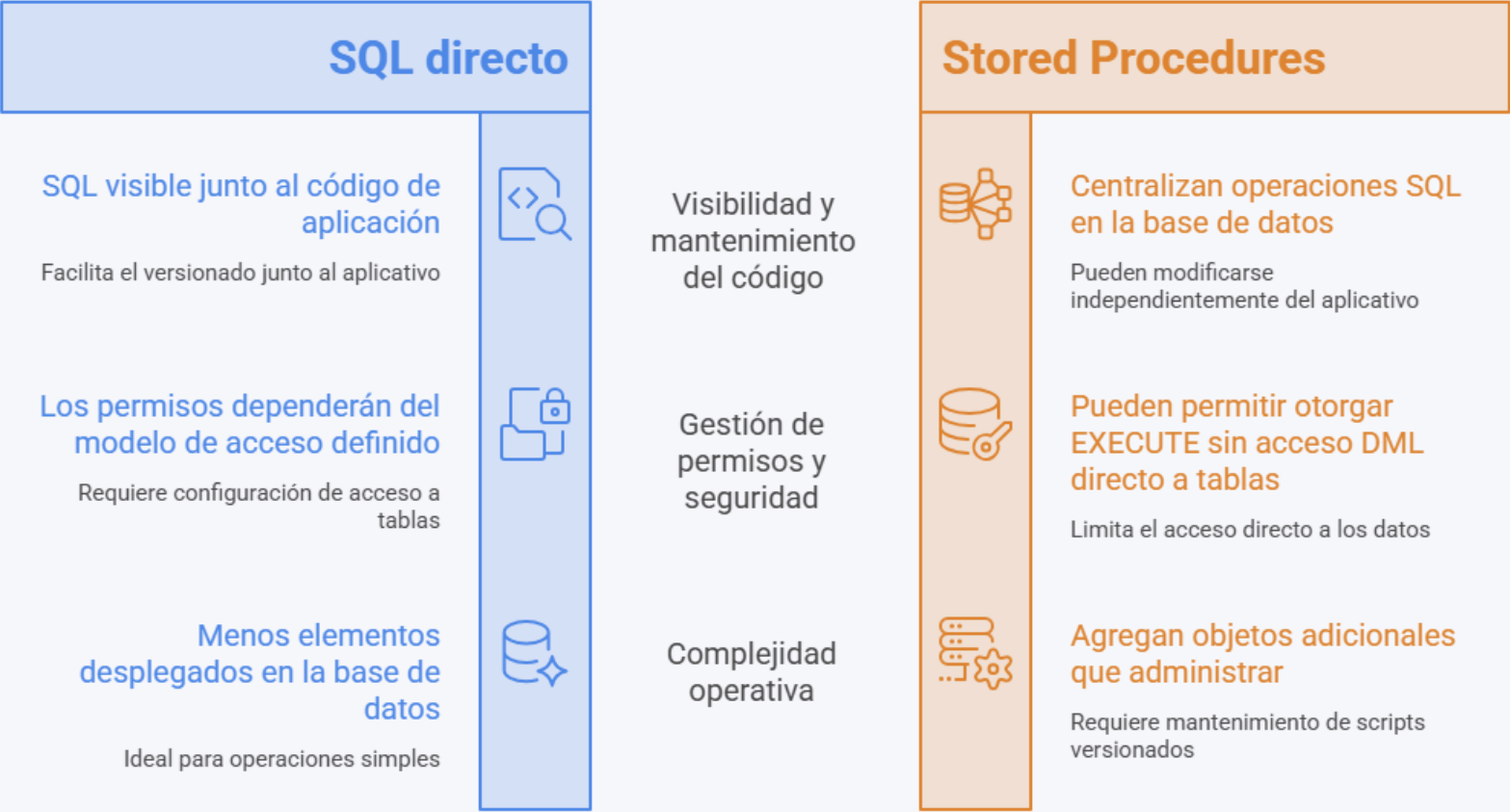
La aplicación invoca una operación definida en la base de datos.

```
new SqlCommand("dbo.usp_Libro_Listar", connection)
{
    CommandType = CommandType.StoredProcedure
};
```

Ambos enfoques deben usar parámetros y ambos terminan ejecutándose en SQL Server.

¿Cuándo usar SQL directo o Stored Procedures?

La elección depende de mantenimiento, permisos, estándares y complejidad.



Mensaje clave Un Stored Procedure no es automáticamente seguro ni automáticamente más rápido. SQL directo parametrizado también evita el patrón típico de SQL Injection.

BiblioNET: caso práctico de la sesión

14

Una aplicación de consola .NET 10 para ver SQL Server y ADO.NET trabajando juntos.

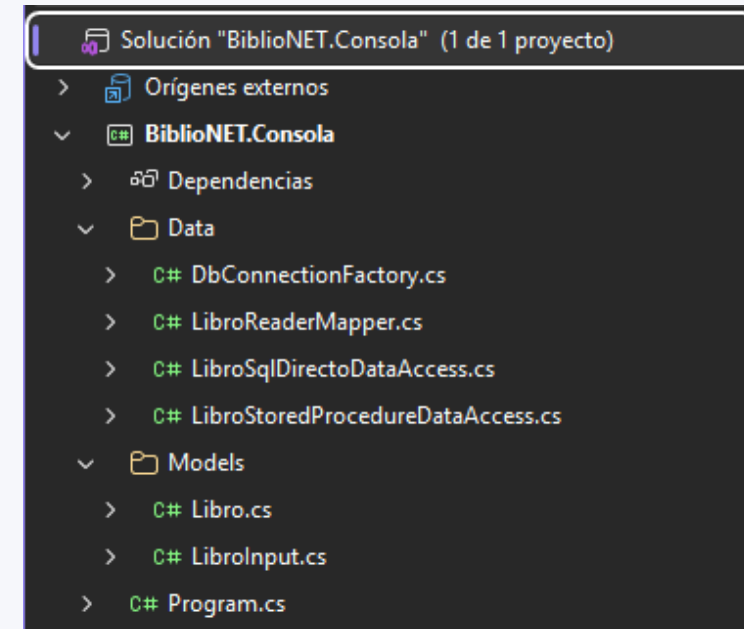
La app permite ejecutar el mismo CRUD de Libros en dos modalidades:

- SQL directo parametrizado
- Stored Procedures

Listar, buscar, insertar, actualizar y eliminar
Validación de resultados desde SSMS

```
=====
                        BiblioNET
=====
Modo de acceso a datos:
1. SQL directo
2. Stored Procedures
0. Salir

Seleccione una opción: |
```



Demo en vivo: recorrido

La presentación explica el patrón; la ejecución muestra el comportamiento real.

Ruta sugerida

- 1 Listar libros con SQL directo y depurar el flujo.
- 2 Repetir la lectura mediante Stored Procedure.
- 3 Insertar un libro desde la consola.
- 4 Validar el registro en SSMS.
- 5 Actualizar el libro y verificar el cambio.
- 6 Eliminar el libro y confirmar el resultado.

Puntos de control

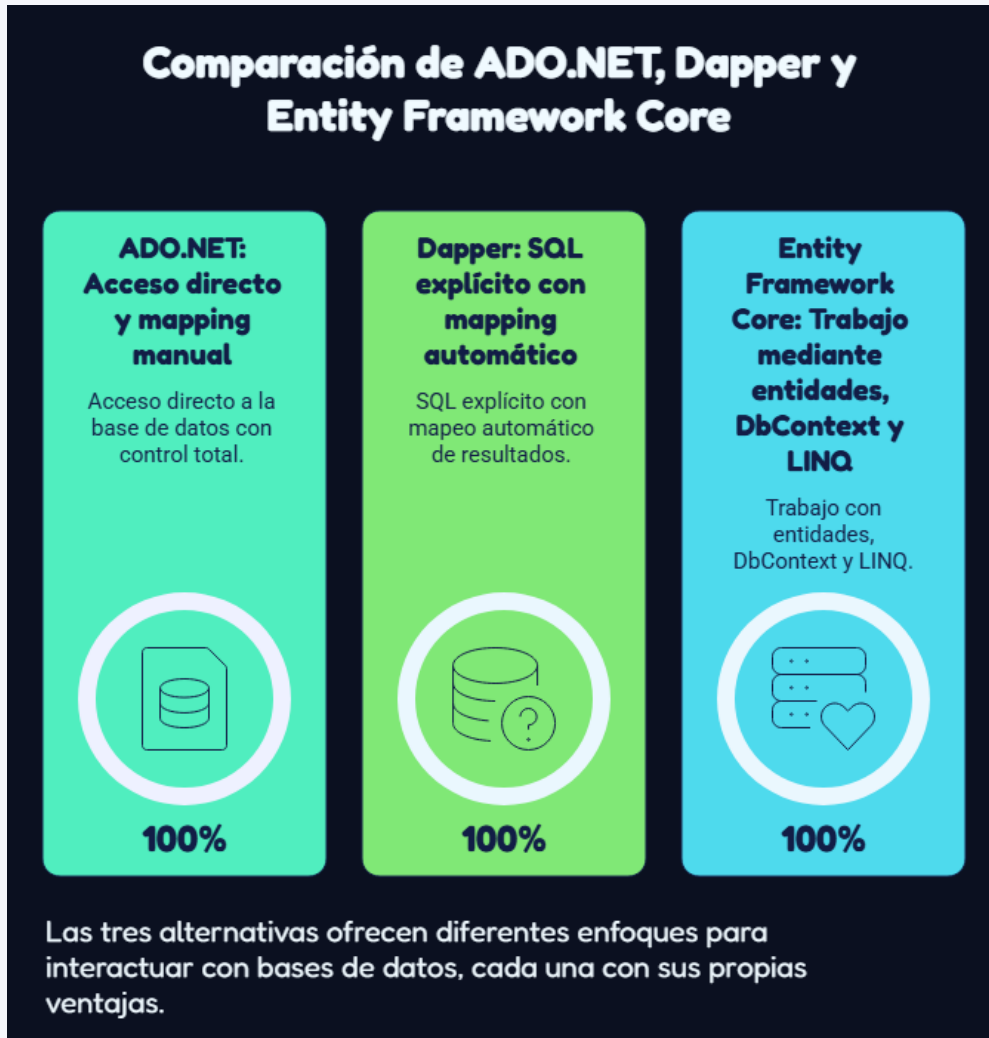
- Breakpoint antes de ejecutar el comando.
- Ver parámetros del SqlCommand.
- Continuar ejecución.
- Consultar tabla en SSMS.
- Explicar el cambio persistido.

No necesitamos capturar todo: el valor está en ver cómo los datos cambian en vivo.

De ADO.NET hacia nuevas abstracciones

16

La siguiente sesión reduce parte del código repetitivo sin perder criterio técnico.



Lo que aprendimos hoy

- Modelar datos en SQL Server.
- Declarar PK, FK, restricciones e índices.
- Abrir conexión desde C#.
- Ejecutar comandos y leer filas.
- Convertir manualmente filas en objetos.

Puente a la sesión 2

Dapper automatiza gran parte del mapping manteniendo SQL explícito. EF Core sube la abstracción hacia entidades, DbContext y LINQ.